

---

# Спецификация

Компонент Eco.Signal.C89

Статус: Черновик

Дата: Ноябрь 8, 2021

Версия: 1.0

---

| Авторы         | Компания |
|----------------|----------|
| Владимир Башев | ПИРФ     |
|                |          |
|                |          |

## Содержание

|                                                     |          |
|-----------------------------------------------------|----------|
| <b>1. Обзор.....</b>                                | <b>3</b> |
| <b>1.1. Введение.....</b>                           | <b>3</b> |
| <b>1.2. Примечание.....</b>                         | <b>3</b> |
| <b>1.3. Ссылки.....</b>                             | <b>3</b> |
| <b>2. Компонент Eco.Signal.C89 .....</b>            | <b>4</b> |
| <b>2.1. IEcoSignalC89 описание на ECO IDL .....</b> | <b>5</b> |
| <b>2.1.1. Функция signal.....</b>                   | <b>6</b> |
| <b>2.1.2. Функция raise.....</b>                    | <b>6</b> |
| <b>Приложение А: Обучающие программы .....</b>      | <b>7</b> |

# 1. Обзор

Данный документ описывает требования к реализации компонента Eco.Signal.C89 (Компонент сигнал).

## 1.1. Введение

Описание.

## 1.2. Примечание

- Ключевые слова в документе

## 1.3. Ссылки

Данный параграф содержит ссылки на информацию, помогающую понять данный документ:

[] – наименование ссылки

Доступен по: <http://адрес>

## 2. Компонент Eco.Signal.C89

Компонент, имеет следующие описание:

## 2.1. IEcoSignalC89 описание на ECO IDL

### ECO IDL

```
import "IEcoBase1.h"

// Signal types
#define SIGINT      2      // interrupt
#define SIGILL      4      // illegal instruction - invalid function image
#define SIGFPE      8      // floating point exception
#define SIGSEGV     11     // segment violation
#define SIGBREAK    21     // Ctrl-Break sequence
#define SIGABRT     22     // abnormal termination triggered by abort call

// Signal action codes
#define SIG_DFL      0      // default signal action
#define SIG_IGN      1      // ignore signal

// Signal error value (returned by signal call on error)
#define SIG_ERR      -1     // signal error value

[
  object,
  uuid(00000000-0000-0000-0000-000053474C31),
]

interface IEcoSignalC89 : IEcoUnknown {

  void*          signal          ([in] int sig,
                                     [in] void (*func)(int));

  int            raise            ([in] int sig);

}
```

### **2.1.1. Функция signal**

Библиотечная функция `C` устанавливает функцию для обработки сигнала, т.е. обработчик сигнала с номером сигнала `sig`.

### **2.1.2. Функция raise**

Библиотечная функция `C` вызывает генерацию сигнала `sig`. Аргумент `sig` совместим с макросами `SIG..`

## Приложение А: Обучающие программы

```

/*
 * <кодировка символов>
 *   Cyrillic (UTF-8 with signature) - Codepage 65001
 * </кодировка символов>
 *
 * <сводка>
 *   EcoSignalC89
 * </сводка>
 *
 * <описание>
 *   Данный исходный файл является точкой входа
 * </описание>
 *
 * <автор>
 *   Copyright (c) 2018 Vladimir Bashev. All rights reserved.
 * </автор>
 *
 */

/* Eco OS */
#include "IEcoSystem1.h"
#include "IdEcoMemoryManager1.h"
#include "IdEcoInterfaceBus1.h"
#include "IdEcoSignalC89.h"
#include "IdEcoStdIOC89.h"
#include "IdEcoStdLibC89.h"
#include "IdEcoThreadManager1.h"

/* Прототипы */
bool_t test_signal(IEcoSignalC89* pISignal, IEcoStdIOC89* pIStdIO, IEcoThread1* pICurrentThread);
bool_t test_raise(IEcoSignalC89* pISignal, IEcoStdIOC89* pIStdIO, IEcoStdLibC89* pIStdLib);

void ECOCDECLMETHOD sighandler(int);

IEcoStdIOC89* g_pIStdIO = 0;

/*
 *
 * <сводка>
 *   Функция EcoMain
 * </сводка>
 *
 * <описание>
 *   Функция EcoMain - точка входа
 * </описание>
 *
 */
int16_t EcoMain(IEcoUnknown* pIUnk) {
    int16_t result = -1;
    /* Указатель на системный интерфейс */
    IEcoSystem1* pISys = 0;
    /* Указатель на интерфейс работы с системной интерфейсной шиной */

```

```

IEcoInterfaceBus1* pIBus = 0;
/* Указатель на интерфейс работы с памятью */
IEcoMemoryAllocator1* pIMem = 0;
char_t* name = 0;
char_t* copyName = 0;
/* Указатель на тестируемый интерфейс */
IEcoSignalC89* pISignal = 0;
/* Указатель на интерфейс стандартной библиотеки */
IEcoStdLibC89* pIStdLib = 0;
/* Указатель на интерфейс стандартного ввода/вывода */
IEcoStdIOC89* pIStdIO = 0;
/* Указатель на интерфейс работы с потоками */
IEcoThreadManager1* pIThreadMgr = 0;
IEcoThread1* pICurrentThread = 0;

/* Проверка и создание системного интерфейса */
if (pISys == 0) {
    result = pIUnk->pVTbl->QueryInterface(pIUnk, &IID_IEcoSystem1, (void **)&pISys);
    if (result != 0 && pISys == 0) {
        /* Освобождение системного интерфейса в случае ошибки */
        goto Release;
    }
}

/* Получение интерфейса для работы с интерфейсной шиной */
result = pISys->pVTbl->QueryInterface(pISys, &IID_IEcoInterfaceBus1, (void **)&pIBus);
if (result != 0 || pIBus == 0) {
    /* Освобождение в случае ошибки */
    goto Release;
}

#ifdef ECO_LIB
/* Регистрация статического компонента для работы со списком */
result = pIBus->pVTbl->RegisterComponent(pIBus, &CID_EcoSignalC89,
(IEcoUnknown*)GetIEcoComponentFactoryPtr_000000000000000000000000053474C31);
if (result != 0) {
    /* Освобождение в случае ошибки */
    goto Release;
}
/* Регистрация статического компонента для работы со стандартным вводом/выводом */
result = pIBus->pVTbl->RegisterComponent(pIBus, &CID_EcoStdIOC89,
(IEcoUnknown*)GetIEcoComponentFactoryPtr_000000000000000000000000053494F31);
if (result != 0) {
    /* Освобождение в случае ошибки */
    goto Release;
}
/* Регистрация статического компонента для работы со стандартной библиотекой */
result = pIBus->pVTbl->RegisterComponent(pIBus, &CID_EcoStdLibC89,
(IEcoUnknown*)GetIEcoComponentFactoryPtr_000000000000000000000000053544431);
if (result != 0) {
    /* Освобождение в случае ошибки */
    goto Release;
}
/* Регистрация статического компонента для работы с менеджером потоков */
result = pIBus->pVTbl->RegisterComponent(pIBus, &CID_EcoThreadManager1,
(IEcoUnknown*)GetIEcoComponentFactoryPtr_B7D8E10E1D3540CCB1CA96F766AF1936);
if (result != 0) {
    /* Освобождение в случае ошибки */
    goto Release;
}

```



```

}
#endif
/* Получение интерфейса управления памятью */
result = pIBus->pVTbl->QueryComponent(pIBus, &CID_EcoMemoryManager1, 0, &IID_IEcoMemoryAllocator1, (void**)
&pIMem);

/* Проверка */
if (result != 0 && pIMem == 0) {
    /* Освобождение системного интерфейса в случае ошибки */
    goto Release;
}

/* Выделение блока памяти */
name = (char_t *)pIMem->pVTbl->Alloc(pIMem, 10);

/* Заполнение блока памяти */
pIMem->pVTbl->Fill(pIMem, name, 'a', 9);

/* Получение интерфейса со стандартным вводом/выводом */
pIBus->pVTbl->QueryComponent(pIBus, &CID_EcoStdIOC89, 0, &IID_IEcoStdIOC89, (void**) &pIStdIO);
if (result != 0 && pIStdIO == 0) {
    /* Освобождение интерфейсов в случае ошибки */
    goto Release;
}
g_pIStdIO = pIStdIO;

/* Получение тестируемого интерфейса */
pIBus->pVTbl->QueryComponent(pIBus, &CID_EcoStdLibC89, 0, &IID_IEcoStdLibC89, (void**) &pIStdLib);
if (result != 0 && pIStdLib == 0) {
    /* Освобождение интерфейсов в случае ошибки */
    goto Release;
}

/* Получение интерфейса для работы с менеджером потоков */
pIBus->pVTbl->QueryComponent(pIBus, &CID_EcoThreadManager1, 0, &IID_IEcoThreadManager1, (void**)
&pIThreadMgr);
if (result != 0 && pIThreadMgr == 0) {
    /* Освобождение интерфейсов в случае ошибки */
    goto Release;
}

/* Получение интерфейса текущего потока */
pICurrentThread = pIThreadMgr->pVTbl->get_CurrentThread(pIThreadMgr);

/* Получение тестируемого интерфейса */
pIBus->pVTbl->QueryComponent(pIBus, &CID_EcoSignalC89, 0, &IID_IEcoSignalC89, (void**) &pISignal);
if (result != 0 && pISignal == 0) {
    /* Освобождение интерфейсов в случае ошибки */
    goto Release;
}

/* Тестирование методов интерфейса */
test_signal(pISignal, pIStdIO, pICurrentThread);
test_raise(pISignal, pIStdIO, pIStdLib);

/* Освобождение блока памяти */

```

```
pIMem->pVTbl->Free(pIMem, name);
```

Release:

```
/* Освобождение интерфейса для работы с интерфейсной шиной */
if (pIBus != 0) {
    pIBus->pVTbl->Release(pIBus);
}

/* Освобождение интерфейса работы с памятью */
if (pIMem != 0) {
    pIMem->pVTbl->Release(pIMem);
}

/* Освобождение тестируемого интерфейса */
if (pISignal != 0) {
    pISignal->pVTbl->Release(pISignal);
}

/* Освобождение интерфейса работы со стандартным вводом/выводом */
if (pIStdIO != 0) {
    pIStdIO->pVTbl->Release(pIStdIO);
}

/* Освобождение интерфейса стандартной библиотеки */
if (pIStdLib != 0) {
    pIStdLib->pVTbl->Release(pIStdLib);
}

/* Освобождение интерфейса для работы с менеджером потоков */
if (pIThreadMgr != 0) {
    pIThreadMgr->pVTbl->Release(pIThreadMgr);
}

/* Освобождение системного интерфейса */
if (pISys != 0) {
    pISys->pVTbl->Release(pISys);
}

return result;
}
```

```
bool_t test_signal(IEcoSignalC89* pISignal, IEcoStdIOC89* pIStdIO, IEcoThread1* pICurrentThread) {
    pISignal->pVTbl->signal(pISignal, SIGINT, sighandler);

    while(1) {
        pIStdIO->pVTbl->printf(pIStdIO, "Going to sleep for a second (Ctrl+C)...\n");
        pICurrentThread->pVTbl->Sleep(pICurrentThread, 1000);
    }

    return 1;
}
```

```
bool_t test_raise(IEcoSignalC89* pISignal, IEcoStdIOC89* pIStdIO, IEcoStdLibC89* pIStdLib) {
    int ret;
```

```
pISignal->pVTbl->signal(pISignal, SIGINT, sighandler);

pIStdIO->pVTbl->printf(pIStdIO, "Going to raise a signal\n");
ret = pISignal->pVTbl->raise(pISignal, SIGINT);

if( ret != 0 ) {
    pIStdIO->pVTbl->printf(pIStdIO, "Error: unable to raise SIGINT signal.\n");
    pIStdLib->pVTbl->exit(pIStdLib, 0);
}

pIStdIO->pVTbl->printf(pIStdIO, "Exiting...\n");

return 1;
}

void ECOCDECLMETHOD sighandler(int signum) {
    g_pIStdIO->pVTbl->printf(g_pIStdIO, "Caught signal %d, coming out...\n", signum);
}
```